

PLC alapismeretek

3. hét – Digitális bemenetek és az első PLC program

Évfolyam / tantárgy	PLC alapismeretek
Óraszám	5 óra (2 óra elmélet + 3 óra gyakorlat)

A hét gondolata

„A PLC nem találgat. Csak azt tudja, amit a bemeneteken keresztül érzékel.”

Tanulási célok

- Megérti a digitális bemenet fogalmát.
- Megkülönbözteti a logikai 0 és 1 állapotot.
- Megismeri a PLC-k bemeneti címezését.
- Elkészíti első működő PLC-programját.
- Megtanulja a program letöltésének és tesztelésének lépéseit.

Bevezetés

A PLC a külvilágról kizárólag a bemeneteken keresztül szerez információt. A nyomógombok és érzékelők jelei alapján hajtja végre a programot.

Mi a digitális jel?

Állapot	Jelentés
0	KI / OFF / Hamis
1	BE / ON / Igaz

A PLC fő részei a következők:

1. **Tápegység (Power Supply)** – biztosítja a PLC belső elektronikájának tápfeszültségét. A PLC típusától függően a betáp lehet például 24 V DC vagy 230 V AC.
2. **CPU – központi feldolgozóegység** – a PLC „agya”. Végrehajtja a felhasználói programot, kezeli a bemeneteket és kimeneteket, valamint a kommunikációt és a diagnosztikát.
3. **Memória** – itt található a PLC operációs rendszere, a felhasználói program és a működés közben használt adatok. Ide tartozhat RAM, Flash és különféle megtartott (retentív) memóriaterület.
4. **Digitális bemenetek (DI)** – kétállapotú jeleket fogadnak, például **0/1**, **KI/BE**. Ilyen jelet adhat egy nyomógomb, végálláskapcsoló vagy közelítésérzékelő.

5. **Digitális kimenetek (DO)** – kétállapotú beavatkozó jeleket szolgáltatnak. Például jelzőlámpát, relét, mágneskapcsolót vagy mágnesszelepet működtethetnek. A kimenet lehet például **relés vagy tranzistoros**.
6. **Analóg bemenetek (AI)** – folytonosan változó mérési értékeket fogadnak, például **0–10 V** vagy **4–20 mA** jelet. Ilyen lehet egy nyomás-, hőmérséklet- vagy szinttávadó jele.
7. **Analóg kimenetek (AO)** – változtatható értékű vezérlőjelet adnak, például **0–10 V** vagy **4–20 mA** jelet egy frekvenciaváltó vagy szabályozószelep számára.
8. **Kommunikációs interfész** – kapcsolatot biztosít más PLC-kkel, HMI-vel, frekvenciaváltóval vagy számítógéppel. Ilyen lehet például **Ethernet, RS-232, RS-485, Modbus vagy PROFINET**.

Érzékelők → BEMENETEK → CPU + MEMÓRIA → KIMENETEK → Beavatkozók

A **tápegység** ellátja a rendszert, a **kommunikációs egység** pedig összeköti a PLC-t más berendezésekkel.

A hardver első alapegysége – a tápegység

Igen. Menjünk végig a **PLC részein egyenként**, és kezdjük a **tápegységgel**.

1. A PLC tápegysége

A tápegység feladata, hogy a PLC elektronikája számára biztosítsa a szükséges, **stabil és megfelelő nagyságú egyenfeszültséget**.

Például egy PLC-re érkezhethet:

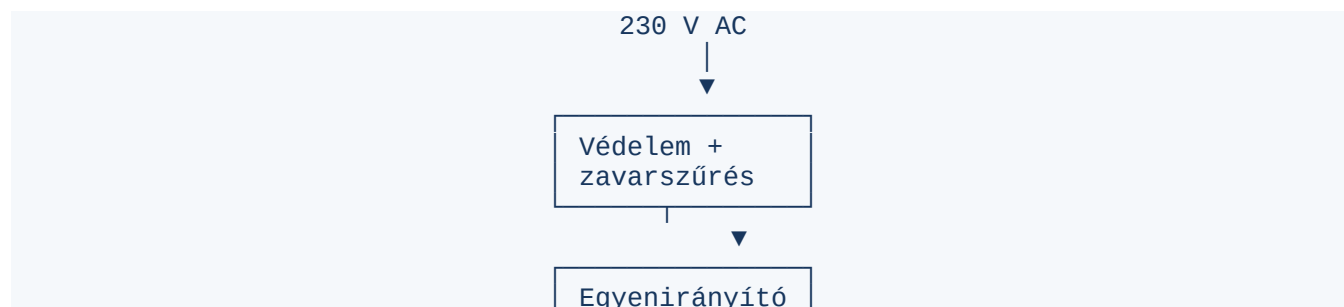
230 V AC → PLC tápegység → kiefeszültségű DC → CPU, memória, elektronika

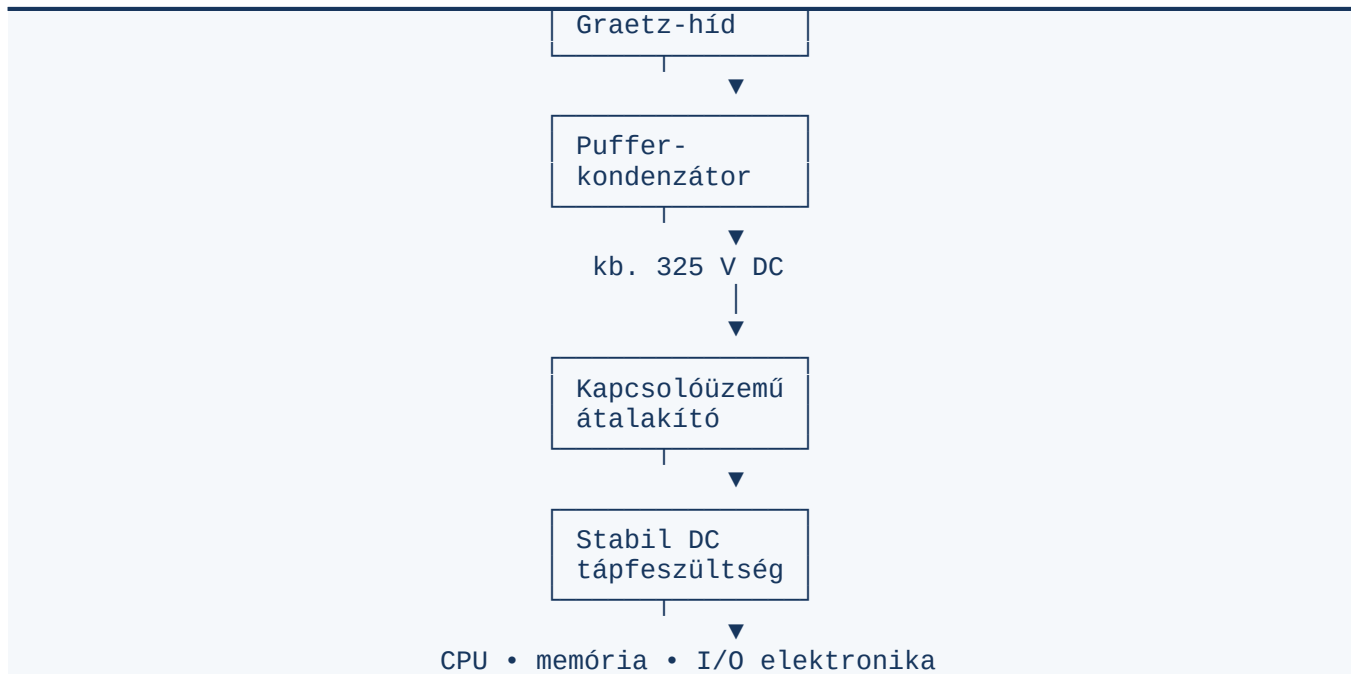
Más PLC-k eleve **24 V DC tápellátást** igényelnek, ilyenkor a hálózati AC/DC átalakítás egy külső tápegységben történik.

Egy 230 V AC-ról működő korszerű PLC-nél leegyszerűsítve:

230 V AC → egyenirányítás → pufferelés → kapcsolóüzemű átalakítás → stabilizálás → DC tápfeszültségek

A folyamatot így képzeld el:





Miért lesz 230 V-ból kb. 325 V?

A hálózati **230 V effektív érték**. A szinuszos feszültség csúcsértéke:

$$U_{\text{csúcs}} = \sqrt{2} \cdot U_{\text{eff}}$$

tehát:

$$230 \cdot 1,414 \approx 325\text{V}$$

A Graetz-híd és a pufferkondenzátor után ezért nagyjából **320–325 V DC** jelenhet meg.

Mi történik ezután?

Ezt a nagy egyenfeszültséget egy **kapcsolóüzemű tápegység** alakítja át. Egy teljesítménytranzisztort – jellemzően MOSFET-et – nagyon gyorsan kapcsolgat:

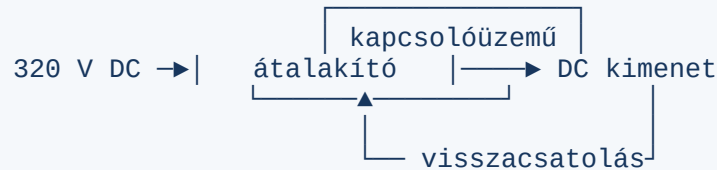


↓
stabil DC

A nagyfrekvenciás transzformátor egyik fontos feladata a megfelelő feszültség szint előállítása, és sok konstrukcióban a **galvanikus leválasztás** biztosítása is.

Honnan tudja, mennyit kell kapcsolni?

Itt jön a **visszacsatolás**. A tápegység folyamatosan figyeli a kimeneti feszültséget:



Ha a kimeneti feszültség **csökken**, a szabályozás több energiát juttat a kimenetre. Ha **növekszik**, kevesebbet.

Ezért marad a PLC tápfeszültsége közel állandó akkor is, amikor a terhelése változik.

PLC-s szempontból miért kritikus?

A CPU **digitális áramkörei nagyon pontos tápfeszültséget igényelnek**. Ha a tápfeszültség leesik vagy instabillá válik, a CPU nem működhet megbízhatóan. Emiatt a PLC-kben rendszerint van **feszültségfelügyelet** is.

Nagyon leegyszerűsítve:

Tápegység rendben → CPU indul → memória/program elérhető → PLC megkezd a működési ciklust.

Innen szerintem logikusan továbbmehetünk a **CPU-val**, de ott érdemes egészen az alapoktól indulni: **órajel → regiszterek → programszámláló → ALU → vezérlőegység**, és ebből felépíteni a Fetch–Decode–Execute működést.

Történelmi nyitó – Neumann János és a Neumann-elv

Neumann János (John von Neumann, 1903–1957) magyar származású matematikus, fizikus és számítástechnikai úttörő volt. Budapesten született, később az Egyesült Államokban dolgozott, többek között a Princetonban működő Institute for Advanced Study professzoraként.

A számítástechnikában azért különösen fontos, mert nevéhez kötjük a **Neumann-architektúra** alapelveit. Ennek egyik legfontosabb gondolata, hogy **a program utasításai és az adatok egyaránt a memóriában tárolhatók**, és a processzor sorban olvassa és hajtja végre az utasításokat.

A klasszikus Neumann-elvű számítógép fő egységei: **központi feldolgozóegység (CPU), memória, bemeneti egység és kimeneti egység**. A CPU-n belül fontos szerepe van a vezérlőegységnek és az aritmetikai-logikai egységnek (ALU).

Ezért kapcsolható jól a **PLC-hez** is. Egy PLC szintén rendelkezik **CPU-val, memóriával, bemenetekkel és kimenetekkel**, és ciklikusan hajtja végre a programját. Fontos azonban, hogy a **PLC scan ciklus nem ugyanaz, mint a klasszikus Neumann-féle fetch–decode–execute utasításciklus**; inkább annak ipari vezérléstechnikai környezetben megjelenő, magasabb szintű működési folyamata.



Tanulóknak egy mondatban:

👉 *Neumann János munkássága alapozta meg a modern, tárolt programú számítógépek egyik meghatározó felépítési elvét – ennek gondolkodásmódja a PLC-k felépítésében is felismerhető.*

A **Neumann-ciklus** — pontosabban **Neumann-elvű működési ciklus** — azt írja le, hogyan hajtja végre egy számítógép vagy mikroprocesszor a memóriában tárolt program utasításait.

A ciklus lényege:

Utasítás beolvasása → dekódolása → végrehajtása → következő utasítás

Egyszerűen:

1. **Fetch – beolvasás:** a processzor kiolvassa a memóriából a következő utasítást.
2. **Decode – értelmezés:** a vezérlőegység megfejti, mit kell csinálni.
3. **Execute – végrehajtás:** például összeadást végez, adatot mozgat, vagy megvizsgál egy feltételt.
4. **Eredmény tárolása:** ha szükséges, az eredményt regiszterbe vagy memóriába írja.
5. **Továbbhaladás:** a programszámláló a következő utasításra mutat, és a ciklus újraindul.

Kapcsolata a PLC-vel

Ez azért érdekes a **PLC oktatásnál**, mert a PLC működése nagyon hasonló, de ott általában **PLC scan cycle**, azaz **PLC ciklus** néven tanítjuk:

Bemenetek beolvasása → PLC-program végrehajtása → kimenetek frissítése → diagnosztika/kommunikáció → új ciklus

Például:

I1 nyomógomb → PLC beolvassa → program vizsgálja I1-et → Q1 bekapcsol → lámpa világít.

A **ciklusidő** pedig az az idő, amennyi alatt a PLC egyszer végigmegy ezen a folyamaton. Ez tipikusan néhány **ms**, de a program és a PLC teljesítménye erősen befolyásolja.

A 3. heti PLC-anyagban ezt érdemes lenne külön kis „**Neumann-ciklus és PLC-ciklus**” résszel kiegészíteni, mert nagyon jól kapcsolódik az első PLC-programhoz.

A **Fetch** (utasítás-beolvasás) a Neumann-elvű processzor működésének első lépése. A szó jelentése egyszerűen: **„hozd el / olvasd be” az utasítást a memóriából.**

Tegyük fel, hogy a memóriában ez található:

Memóriací m	Tartalo m
100	LOAD A
101	ADD B
102	STORE C

A processzorban van egy **PC (Program Counter – programszámláló)**. Ez tartalmazza annak a memóriahelynek a címét, **ahol a következő végrehajtandó utasítás található.**

Ha például:

PC = 100

akkor a Fetch során leegyszerűsítve ez történik:

PC → cím a memóriának → memória kiolvasása a 100. helyet → „LOAD A” → utasításregiszter (IR)

Vagyis:

PC → Memória → IR

Ezután a programszámláló továbblép:

PC: 100 → 101

Most következhet a **Decode**, amikor a CPU értelmezi, hogy mit jelent a beolvasott LOAD A utasítás, majd az **Execute**, amikor végre is hajtja.

Tehát a teljes kör:

FETCH → DECODE → EXECUTE → FETCH → DECODE → EXECUTE → ...

PLC-s párhuzam

A PLC-nél két szintet érdemes megkülönböztetni. A PLC CPU-ja belül ugyanúgy processzorutasításokat hajt végre, tehát ott is történik **fetch–decode–execute**. A PLC felhasználója viszont inkább a **PLC scan ciklust** látja:

Bemenetek beolvasása → PLC-program végrehajtása → kimenetek frissítése → új ciklus

Ezért például az általad használt I1 → Q1 program végrehajtása mögött rengeteg nagyon gyors **Fetch–Decode–Execute** művelet történik.

Bemeneti címezés

- Eaton easy: I1, I2...
- LG: P000, P001...
- Omron: 0.00, 0.01...

Az első PLC-program

Feladat: START nyomógomb megnyomására világítson a jelzőlámpa.

|----[I1]------(Q1)----|

Hogyan működik a program?

Jelentése: **ha I1 = 1, akkor Q1 = 1; ha I1 = 0, akkor Q1 = 0.**

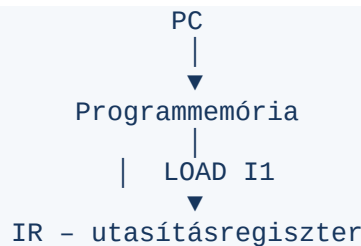
Fontos, hogy a PLC-program $I1 \rightarrow Q1$ utasítása a CPU-ban nem feltétlenül **egyetlen gépi utasítás**. A PLC futtatórendszere ezt több processzorutasításra fordíthatja. Oktatási célra azonban nagyon jól szemléltethető a Fetch–Decode–Execute működés.

1. FETCH – utasítás beolvasása

Tegyük fel, hogy a következő végrehajtandó utasítás:

```
LOAD I1
```

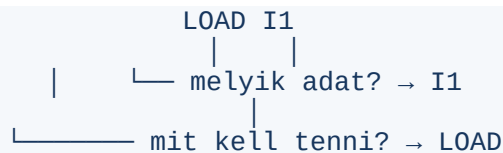
A **PC (Program Counter)** megmondja, hol található ez az utasítás a memóriában.



A CPU tehát **beolvassa** a `LOAD I1` utasítást.

2. DECODE – értelmezés

A vezérlőegység megvizsgálja a beolvasott utasítást:



A CPU lényegében azt „érti meg”:

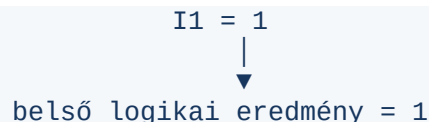
„Olvasd ki az I1 bemenet állapotát.”

Tegyük fel:

```
I1 = 1
```

3. EXECUTE – végrehajtás

A CPU végrehajtja az utasítást:



Ezután jöhet a következő utasítás.

Újabb FETCH

A PC már a következő utasításra mutat:

```
STORE Q1
```

A CPU ezt is beolvassa:

FETCH: STORE Q1

DECODE: „Az aktuális logikai eredményt Q1-hez kell rendelni.”

EXECUTE:

```
logikai eredmény = 1
    ↓
    Q1 = 1
```

Így a teljes folyamat oktatási modellben:

```

                                I1 → Q1
                                ↓
                                FETCH    LOAD I1
                                ↓          ↓
                                DECODE    „Olvasd I1-et!”
                                ↓          ↓
                                EXECUTE    I1 = 1
                                ↓          ↓
                                FETCH    STORE Q1
                                ↓          ↓
                                DECODE    „Írd az eredményt Q1-be!”
                                ↓          ↓
                                EXECUTE    Q1 = 1

```

Tehát ha a tanuló megnyomja az **I1-re kötött START nyomógombot**, a végeredmény:

I1 = 1 → program feldolgozza → Q1 = 1 → jelzőlámpa bekapcsol.

Laborfeladat

- Nyomógomb bekötése az első bemenetre.
- Jelzőlámpa bekötése az első kimenetre.
- Program elkészítése és letöltése.
- Online monitor használata.

Gyakori hibák

- Hibás címezés.
- STOP mód.
- Hiányzó tápfeszültség.
- Hibás bekötés.

- A program nincs letöltve.

Mérnöki gondolkodás

A működő LED még nem jelenti azt, hogy a vezérlés biztonságos. Mindig vizsgáljuk meg a rendszer viselkedését rendellenes helyzetekben is.

IAP Extra

Az online monitor valós időben mutatja a bemenetek, kimenetek és a program állapotát.

Összefoglalás

A digitális bemenetek jelentik a PLC kapcsolatát a külvilággal, ezek alapján működik a program.

Önellenőrző kérdések

1. Mit jelent a digitális jel?
2. Mit jelent az I1 és a Q1?
3. Mi a különbség a 0 és az 1 között?
4. Miért fontos az online monitor?
5. Sorolj fel négy gyakori hibát!

Idézet

*„A PLC nem gondolkodik helyettünk – csak pontosan végrehajtja azt, amit megterveztünk.”
– IAP mérnöki alapelv*